

Аудит кода и архитектурные рекомендации

Проект: Workout Bot (4G) | Дата: 16 апреля 2026

1. Файл: `backend/api/routers/auth.py`

Проблема: Дублирование логики (Violating DRY Principle)

Логика вычисления `compat_role` и `effective_onboarding_done` идентична в эндпоинтах `/telegram` и `/register`.

Что изменить:

Вынести расчет роли и статуса онбординга в единую внутреннюю функцию `_resolve_user_permissions`.

Почему это важно:

При добавлении новых условий доступа или новых ролей (например, "Куратор"), вам придется менять код в двух местах. Это неизбежно приведет к рассинхронизации и багам авторизации.

Выгода: Гарантированная консистентность прав доступа во всем приложении и сокращение объема кода на 15-20%.

2. Файл: backend/core/deps.py

Проблема: Побочные эффекты в зависимостях (Side Effects in Dependencies)

Функция `get_current_user` выполняет UPDATE запрос к базе данных, если промокод игрока истек. Это происходит при каждом запросе пользователя.

Что изменить:

Зависимость должна только проверять доступ. Логику "самоисцеления" (проставление `deactivated_at`) следует вынести в фоновую задачу (Background Task) или отдельный сервис.

Почему это важно:

GET-зависимость должна быть максимально быстрой и "чистой". Запись в БД при каждой проверке токена создает лишнюю нагрузку на PostgreSQL и увеличивает время отклика API (Latency).

Выгода: Повышение производительности API и четкое разделение ответственности (Separation of Concerns).

3. Файл: frontend/src/hooks/useAuth.ts

Проблема: Хрупкое управление состоянием (State Management Fragility)

Использование `setInterval` для поллинга (опроса) статуса фотографий может привести к накоплению запросов, если сервер отвечает медленнее, чем заданный интервал.

Что изменить:

Заменить `setInterval` на рекурсивный `setTimeout` или использовать специализированные библиотеки (например, `React Query`), которые умеют делать умный поллинг.

Почему это важно:

Это предотвратит "заспамливание" сервера запросами на медленных соединениях и исключит риск обновления состояния компонента после его размонтирования.

Выгода: Стабильный интерфейс без "дерганий" и лишнего сетевого трафика.

4. Файл: 200_workoutSessionMachine.ts

Проблема: Магические числа и отсутствие обработки сетевых сбоев

Количество упражнений (16) жестко прописано в коде. В блоке `invoke` нет явной логики повторных попыток при сбое загрузки видео.

Что изменить:

Вынести конфигурацию тренировки в контекст машины. Добавить промежуточное состояние `retryingUpload` для обработки ошибок сети.

Выгода: Возможность легко менять программу тренировок без переписывания логики FSM и высокая отказоустойчивость при плохом интернете.

Заключение: Код написан на высоком уровне с использованием современных практик. Предложенные изменения направлены на переход от стадии "MVP / Прототип" к стадии "Production Ready / Масштабируемый продукт".